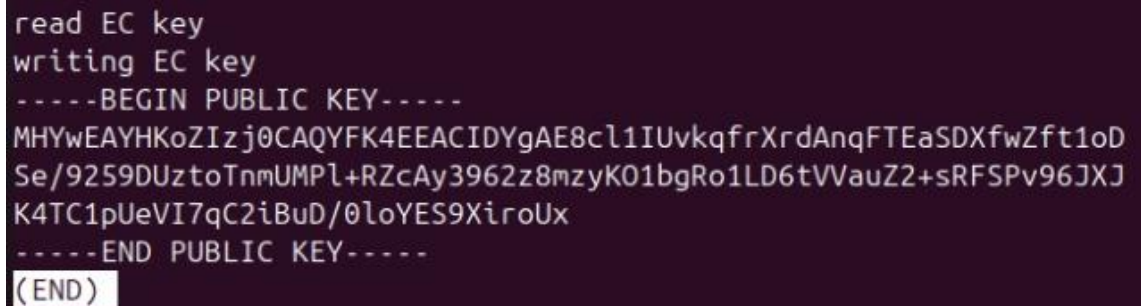


Lab 09 Template

Some questions require multiple parts to be answered, be sure to discuss them in full.

Part 01:

1. **Screenshot of the output from** `openssl ec -text -in <netid>_private_key.pem |less`
(5 points)



```
read EC key
writing EC key
-----BEGIN PUBLIC KEY-----
MHYwEAYHKoZIzj0CAQYFK4EEACIDYgAE8cl1IUvkqfrXrdAnqFTEaSDXfwZft1oD
Se/9259DUztoTnmUMPl+RZcAy3962z8mzyKO1bgRo1LD6tVVauZ2+sRFSPv96JXJ
K4TC1pUeVI7qC2iBuD/0loYES9XiroUx
-----END PUBLIC KEY-----
(END)
```

2. **What is the difference between the two curve identifiers?**
(5 points)

The NIST curve name (P-384) and the ASN1 OID (secp384r1) both basically refer to the same elliptic curve, but they are used in different contexts. The NIST name is simpler and easier to read by humans, which is assigned by NIST for easier reference, whereas the ASN1 OID is a standardized identifier, which is used in cryptographic protocols and encoding formats in order to uniquely specify the curve.

3. **What does the CONNECTED(00000003) line indicate about the steps that occurred before establishing the SSL/TLS handshake?**
(5 points)



```
CONNECTED(00000003)
Sent Record
Header:
```

Over here, this line basically indicates that a TCP connection between the client and the server was successfully established before the TLS handshake began, which means that the 3-way handshake completed which allows the TLS handshake to start over an already established connection.

4. **Take a screenshot of the client key_share information**

```

extension_type=key_share(51), length=103
  NamedGroup: secp384r1 (P-384) (24)
  key_exchange: (len=97): 04C8FF57212C0AE6AC1BC8347890761E0C28E511F8D0CDC6D8767B27EDDF9B3AABE509C13CD8
8BDD911093B7F67E1531A33D2B07B42D631FE812A046730C051CDAC4FA0238CD06815D87AED1858CE959F69BAEA5EFBF8799E7D5E15DF17

```

(5 points)

- What information do you think the client `key_share` fields are providing to the server? How might the server use this information during the TLS 1.3 handshake to establish a secure connection? Does this contribute to perfect forward secrecy? Why or why not?

(10 points total, 2.5 points each question)

Over here, the `key_share` field contains the client's ephemeral public key along with the elliptic curve that is basically used (secp384r1), which basically allows the server to combine the client's public key with its own private key to generate a shared secret. Also, the server sends its own `key_share` values, where both sides basically compute the same secret independently without sending it over the network, which basically contributes to the perfect forward secrecy because the keys are temporary and unique for each session, so even if long-term keys are compromised, where past sessions remain secure.

- Take a screenshot of the supported version extension for the server. What version(s) of TLS does the server actually support?

(10 points total, 5 points screenshot, 5 points question)

```

ServerHello, Length=183
  server_version=0x303 (TLS 1.2)
  Random:
    gmt_unix_time=0x2E0495E7
    random_bytes (len=28): 7E0DCAB9C32E13396FCC0B5CFF2FC31809D6C048D70E3B31AD3D0F68
  session_id (len=32): E8EBB6399ABDB0DA1DDB3BDC23457E334696A0F07EF8E0B5171B67F84D2D50C8
  cipher_suite {0x13, 0x02} TLS_AES_256_GCM_SHA384
  compression_method: No Compression (0x00)
  extensions, length = 111
    extension_type=supported_versions(43), length=2
      TLS 1.3 (772)

```

Over here, from the screenshot we can basically see that the server basically supports TLS 1.3, which is shown in the `supported_versions` extension in the server hello message. It also shows that the `server_version` field displays TLS 1.2 for backward compatibility, but the actual negotiated version is TLS 1.3.

- Take a screenshot of the `key_share` information.

(5 points)

```

extension_type=key_share(51), length=101
  NamedGroup: secp384r1 (P-384) (24)
  key_exchange: (len=97): 04A46C5BFD098A38626DB4056C30E0E14846E8DEB61C72A4ACFF23018B82BE0C48BD46939DFC169BA
F685B7B198FBC09CEB43B2648713B98E6D341736640358C946972DA3D19063D7700EB402C4D36060825BBA4A0967872FDD4EF6610303A00

```

8. **Take a screenshot of the certificate chain**
(5 points)

```
Certificate chain
0 s:C = US, ST = Iowa, L = Ames, O = 3310.com, OU = homework, CN = openssl.homework.3310.com
i:C = US, ST = Iowa, L = Ames, O = 3310.com, OU = homework, CN = ecdhecerts.homework.3310.com, emailAddress = ecdhece
rts.homework.3310.com
a:PKEY: id-ecPublicKey, 384 (bit); sigalg: ecdsa-with-SHA256
v:NotBefore: Oct 30 03:30:44 2024 GMT; NotAfter: Oct 28 03:30:44 2034 GMT
-----BEGIN CERTIFICATE-----
MIIDAzCCAoigAwIBAgICEAkwwCgYIKoZIzj0EAwIwGAUxCzAJBgNVBAYTA1VTMQ0w
CwYDVQQIDARJb3dhM0QwCwYDVQQHDA RBbWVzMREwDwYDVQQKDAgzMzEwLmNvbTER
MA8GA1UECwwlIG9tZXdtvcmsxJTAAJBGNVBAAMMHGVjZGhlY2VydmHMuaG9tZXdtvcmsu
MzMcMC5jb2B0eXkzApBgqhkiG9w0BCQEHEHVjZGhlY2VydmHMuaG9tZXdtvcmsuMzMc
MC5jb2B0eXkzNmQxMDMwMDMzMDOQWhcNMzQxMDI4MDMzMDOQWajB1MQswCQYDVQVQMG
EWJVVUZENMASGA1UECAESW93YTENMASGA1UEBwwEQWllczERMABGUA1UECGwIMzIx
MC5jb2B0eXkzETAPBgNVBASMCChvbwV3b3JrMSIWIAYDVQQDDBlvcGUC3NsLmhvbwV3
b3JrLjMzMtAUy29tMHYwEAYHKoZIzj0CAQYFK4EEACIDYgAEZhnw/oMJ3VrUWeXq
tLES+r1Je3NSfWaoI3VYylqrZ7wpApETWLrT+lrIFR1VFyy720IScZZTP/tUT4P0
DJtdDJ7KNB8PSa5mm9+rSDkfwe2tlqv13iuAEPdRtuuBZJ1so4G5MIG2MASGA1Ud
DGWEAWIEMDATBgNVHSUEDAKBggrBgEFBQcDATBSBGNVHRESszBJhwQiEGwfghlv
cGUC3NsLmhvbwV3b3JrLjMzMtAUy29tgh1d3dcub3lbnNzbC5ob21ld29ay4z
MzEwLmNvbTYHb3B1bnNzbDADBgNVHQ4EFggQUQUJK2kzCtkZHrgQoI+F5d1teuCuw
HWYDVR0jBBgwFoAUmN/GzDIIsrBwkZTQ2xsHnMbM0d4wCgYIKoZIzj0EAwIDAQAQ
WagIA0YjncCRxqQhzBGVfvqxS/BExNEZshzyoILesIbxXJNvnseGOjvV5ondot3
X602uwIXAIUF0sNue4W9FSrkZ4IQ0GL+hZXiutcvGWKqwyVa88ToR3LkLVFUS5Qd
WdlKz7FXQ==
-----END CERTIFICATE-----
1 s:C = US, ST = Iowa, L = Ames, O = 3310.com, OU = homework, CN = ecdhecerts.homework.3310.com, emailAddress = ecdhece
rts.homework.3310.com
i:C = US, ST = Iowa, L = Ames, O = 3310.com, OU = homework, CN = ecdhecerts.homework.3310.com, emailAddress = ecdhece
rts.homework.3310.com
a:PKEY: id-ecPublicKey, 384 (bit); sigalg: ecdsa-with-SHA256
v:NotBefore: Oct 20 00:35:17 2024 GMT; NotAfter: Oct 23 00:35:17 2044 GMT
```

9. Look at the subject (s) and the issuer (i) for each of the two certificates. **Which number is the certificate for the Certificate Authority (CA)? Why?**
(10 points total, 5 points for each question)

Over here, certificate 1 is the Certificate Authority (CA) since its self-aligned as its subject and issuer are same and certificate 0 is the server certificate, and its issuer basically matches the subject of certificate 1, which means that it has been signed by the CA.

10. What happens if the server's certificate validation fails at any depth?
(5 points)

If in any case the server's certificate validation fails at any depth, the TLS handshake will basically fail as well and the connection will be terminated, so the client will not trust the server, which in turn will prevent any secure communication.

11. **Expand the acronym for each piece of the cipher suite and explain the purpose of each.**
(10 points total, 2 points each piece)

```

SSL handshake has read 2255 bytes and written 2006 bytes
Verification: OK
--I
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 384 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
read from 0x64b35113ac70 [0x64b351148823] (5 bytes => 5 (0x5))
0000 - 17 03 03 03 ca          .....
Received Record
Header:
  Version = TLS 1.2 (0x303)
  Content Type = ApplicationData (23)
  Length = 970
read from 0x64b35113ac70 [0x64b351148828] (970 bytes => 970 (0x3CA))
0000 - 06 79 0e a5 06 db 19 06-60 94 c0 ce 81 c7 e3 e3   .y.....'.....
0010 - a0 b5 7f a8 3e 56 29 3d-bf 7a f8 9a d4 7b f5 25   ....>V)=.z...{.%
0020 - 49 40 97 17 68 2e ff 48-ab 5c ea a4 3c a9 e7 2a   I@..h..H.\..<...*
0030 - 3c 0e ee e1 ce 8f a4 dd-8a 61 a8 76 ae 7c 4f b5   <.....a.v.|0.
0040 - e8 d3 86 5b bd b0 ef 51-66 df 08 97 9a 56 33 7d   ...[...Qf....V3}
0050 - 52 8b 08 f2 7b 61 bf c2-da 9a 40 f0 62 7c 87 39   R...{a....@.b|.9
0060 - 4c 38 ba af df 1b c4 39-41 60 de cb 57 31 6c 4f   L8.....9A`..W1l0
0070 - 0c b9 5c 43 7d e6 89 cf-22 80 b5 69 90 9e eb 14   ..\C}..."..i....
0080 - ee de 13 25 a5 3d 2c 23-75 ff 87 1d bf 30 df 17   ...%.=#u....0..
0090 - 98 66 0a 08 d0 ce 77 5e-f9 66 be 2c e5 6a 13 1a   .f....w^.f.,.j..
00a0 - 85 e9 72 4d 68 98 32 0f-5e 65 11 eb 82 44 6e 8b   ..rMh.2.^e...Dn.
:

```

TLS = Transport Layer Security, which is basically used to establish a secure connection.

AES_256 = It is the Advanced Encryption Standard, which has a 256-bit key for symmetric encryption.

GCM = Galois/Counter Mode. This basically provides both encryption and integrity.

SHA384 = Secure Hash Algorithm which is used for message authentication and integrity.

12. What does “getting new key material” mean? (5 points)

Getting new key material basically means that fresh cryptographic keys are generated for each session, even when a session is resumed using a session ticket. In TLS 1.3, the session tickets basically allow faster reconnection by basically using a PSK handshake, however new keys are still derived using fresh

randomness. This basically ensures that previous session keys cannot be reused or compromised, which basically maintains the perfect forward secrecy.

13. Why would creating pkcs12 files be a bad idea generally speaking?
(5 points)

Creating these files could be risky since it can store both the private key and the certificate together in a single file, so if this file is compromised, an attacker can basically gain access to the private key, which allows them to basically impersonate the user or decrypt secure communications and additionally, the PKCS files are only protected by a password, which may be weak or could be cracked easily. Moreover, since the file is portable and easy to copy, it also creates a single point of failure and increases the risk for unauthorized access.

14. Take a screenshot of the ciphers supported in s_server binary.
(5 points)

```
s_server -accept 4433 -cert /home/cpre3310/openssl_certificate.pem -key /home/cpre3310/private/openssl_private_key.pem -CAfile /home/cpre3310/ca_cert.pem -Verify 1 -w
Secure Renegotiation IS NOT supported
Ciphers supported in s_server binary
TLSv1.3 : TLS_AES_256_GCM_SHA384 TLSv1.3 : TLS_CHACHA20_POLY1305_SHA256
TLSv1.3 : TLS_AES_128_GCM_SHA256 TLSv1.2 : ECDHE-ECDSA-AES256-GCM-SHA384
TLSv1.2 : ECDHE-RSA-AES256-GCM-SHA384 TLSv1.2 : DHE-RSA-AES256-GCM-SHA384
TLSv1.2 : ECDHE-ECDSA-CHACHA20-POLY1305 TLSv1.2 : ECDHE-RSA-CHACHA20-POLY1305
TLSv1.2 : DHE-RSA-CHACHA20-POLY1305 TLSv1.2 : ECDHE-ECDSA-AES128-GCM-SHA256
TLSv1.2 : ECDHE-RSA-AES128-GCM-SHA256 TLSv1.2 : DHE-RSA-AES128-GCM-SHA256
TLSv1.2 : ECDHE-ECDSA-AES256-SHA384 TLSv1.2 : ECDHE-RSA-AES256-SHA384
TLSv1.2 : DHE-RSA-AES256-SHA256 TLSv1.2 : ECDHE-ECDSA-AES128-SHA256
TLSv1.2 : ECDHE-RSA-AES128-SHA256 TLSv1.2 : DHE-RSA-AES128-SHA256
TLSv1.0 : ECDHE-ECDSA-AES256-SHA TLSv1.0 : ECDHE-RSA-AES256-SHA
SSLv3 : DHE-RSA-AES256-SHA TLSv1.0 : ECDHE-ECDSA-AES128-SHA
TLSv1.0 : ECDHE-RSA-AES128-SHA SSLv3 : DHE-RSA-AES128-SHA
TLSv1.2 : RSA-PSK-AES256-GCM-SHA384 TLSv1.2 : DHE-PSK-AES256-GCM-SHA384
TLSv1.2 : RSA-PSK-CHACHA20-POLY1305 TLSv1.2 : DHE-PSK-CHACHA20-POLY1305
TLSv1.2 : ECDHE-PSK-CHACHA20-POLY1305 TLSv1.2 : AES256-GCM-SHA384
TLSv1.2 : PSK-AES256-GCM-SHA384 TLSv1.2 : PSK-CHACHA20-POLY1305
TLSv1.2 : RSA-PSK-AES128-GCM-SHA256 TLSv1.2 : DHE-PSK-AES128-GCM-SHA256
TLSv1.2 : AES128-GCM-SHA256 TLSv1.2 : PSK-AES128-GCM-SHA256
TLSv1.2 : AES256-SHA256 TLSv1.2 : AES128-SHA256
TLSv1.0 : ECDHE-PSK-AES256-CBC-SHA384 TLSv1.0 : ECDHE-PSK-AES256-CBC-SHA
SSLv3 : SRP-RSA-AES-256-CBC-SHA SSLv3 : SRP-AES-256-CBC-SHA
TLSv1.0 : RSA-PSK-AES256-CBC-SHA384 TLSv1.0 : DHE-PSK-AES256-CBC-SHA384
SSLv3 : RSA-PSK-AES256-CBC-SHA SSLv3 : DHE-PSK-AES256-CBC-SHA
SSLv3 : AES256-SHA TLSv1.0 : PSK-AES256-CBC-SHA384
SSLv3 : PSK-AES256-CBC-SHA TLSv1.0 : ECDHE-PSK-AES128-CBC-SHA256
TLSv1.0 : ECDHE-PSK-AES128-CBC-SHA SSLv3 : SRP-RSA-AES-128-CBC-SHA
SSLv3 : SRP-AES-128-CBC-SHA TLSv1.0 : RSA-PSK-AES128-CBC-SHA256
TLSv1.0 : DHE-PSK-AES128-CBC-SHA256 SSLv3 : RSA-PSK-AES128-CBC-SHA
SSLv3 : DHE-PSK-AES128-CBC-SHA SSLv3 : AES128-SHA
TLSv1.0 : PSK-AES128-CBC-SHA256 SSLv3 : PSK-AES128-CBC-SHA
---
Ciphers common between both SSL end points:
TLS_AES_128_GCM_SHA256 TLS_CHACHA20_POLY1305_SHA256 TLS_AES_256_GCM_SHA384
ECDHE-ECDSA-AES128-GCM-SHA256 ECDHE-RSA-AES128-GCM-SHA256 ECDHE-ECDSA-CHACHA20-POLY1305
ECDHE-RSA-CHACHA20-POLY1305 ECDHE-ECDSA-AES256-GCM-SHA384 ECDHE-RSA-AES256-GCM-SHA384
ECDHE-ECDSA-AES256-SHA ECDHE-ECDSA-AES128-SHA ECDHE-RSA-AES128-SHA
ECDHE-RSA-AES256-SHA AES128-GCM-SHA256 AES256-GCM-SHA384
AES128-SHA AES256-SHA
```

15. Find and record in your lab report at least one that should no longer be used in the real world. Support your reason why it shouldn't be used.
(10 points total, 5 points each answer)

One insecure protocol shown in the screenshot for the previous question is SSLv3, which should no longer be used because it is vulnerable to well-known attacks, such as the POODLE attack, which allows attackers to decrypt encrypted data. SSLv3 also uses some outdated cryptographic mechanisms and lacks modern security protections found in newer protocols like TLS 1.2 and TLS 1.3. Due to these vulnerabilities, SSLv3 has been deprecated and is considered insecure for any real-world use.